Ejercicio 1

```
//class Relation
public class Relation<T, R> {

    private T entity1;
    private R entity2;
    private String description;

    public Relation (T entity1, R entity2, String description
        this.entity1 = entity1;
        this.entity2 = entity2;
        this.description = description
    }

    public T getEntity1() {
        return entity1;
    }

    public void setEntity1(T entity1) {
        this.entity1 = entity1;
    }

    public R getEntity2() {
        return entity2;
    }

    public void setEntity2(R entity2) {
        this.entity2 = entity2;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }
```

```
// tomando en cuenta el Relation.java
import java.util.*

public class Prueba {
  public static void main(String[] args) {
    Relation<String, String> r1 = new Relation<>(
      "este cachorro",
      "esta señora",
      "es mascota de"
    );

    List<String> empresas = Arrays.asList("Empresa A", "Empresa B", "Empresa C");
    Relation<String, List<String>> r2 = new Relation<>(
      "este empresario",
      empresas,
      "posee"
    );

    Relation<String, String> r3 = new Relation<>(
      "esta ciudad",
      "este pais",
      "es la capital de"
    );

    Relation<String, String> r4 = new Relation<>(
      "este veterinario",
      "este cachorro",
      "atiende a"
    );

    Relation<String, String> r5 = new Relation<>(
      "este aviones",
      "esta ciudad",
      "se dirige a"
    );

    imprimirRelacion(r1); //Método implementado (función)
    imprimirRelacion(r5);
  }

  public static void imprimirRelacion(Relation<?, ?> r) {
    System.out.println(r.getEntity1() + " " + r.getDescripcion + " " + r.getEntity2());
  }
}
```

para verificar si acepta cualquier tipo de dato

Ejercicio 2

Implementación 1

$\log(n)$

1000

$27\log(n)$

A priori, $\log(n) + 1000 + 27\log(n) = 28\log(n) + 1000$.

El 28 se descarta por ser coeficiente

$= \log(n) + 1000$

$= O(\log(n)) + O(1)$

$= O(\log(n))$

Es decir

$\log(n) > 1000$

$\Rightarrow O(\log(n)) + O(1) + O(27\log(n)) + O(\log(n))$

$= O(\log(n))$

Implementación 2

$n\log(n)$

$15n$

$0{,}002n^2$

El peor de los casos está dado por el valor más alto, note que

$15n = n$, $0{,}002n^2 = n^2$

$\Rightarrow \underbrace{n^2}_{\text{mayor}} > n\log(n) > n$

Entonces

$= O(n^2)$ ($O(n)$)

$= O(\log(n)) + O(n) + O(n^2)$

Implementación 3

$37n$

$n\log(n^2)$

$5000\log(n)$

A priori,

$37n = n$, $5000\log(n) = \log(n)$, $n\log(n^2) = n\log(n)$

Entonces

$\Rightarrow \underbrace{n\log(n^2)}_{\text{mayor}} > n > \log(n)$

$= O(n\log(n^2))$

$\Rightarrow O(n) + O(2n\log(n)) + O(n)$

$= O(n\log(n^2))$

Implementación 4

$1000n^2$

$16n$

2^n

A priori,

$1000n^2 = n^2$, $16n = n$

$\Rightarrow 2^n > 1000n^2 > 16n$

$O(1000n^2) + O(16n) + O(2^n)$

$= O(2^n)$

Implem. 5

2^{10}

35

1000000

$2^{10} + 35 + 1000000 = O(1)$